

raft-kv

A Distributed Key-Value Store with Raft Consensus,
Deterministic Simulation, and a Linearizability Checker

Deep Dive: an exhaustive, first-principles walkthrough

Read this first: what the project is, in one paragraph

raft-kv is a database that stores simple key-value pairs (like `city = lahore`) and keeps identical copies of that data on several computers at once, so that the data survives any one of them dying. Keeping several computers in exact agreement, while they crash and their network breaks, is the problem called *consensus*. This project solves it with an algorithm called *Raft*, written from scratch in Python with no consensus libraries. But the code that solves consensus is only about a third of the project. The other two thirds are the interesting part: a *simulator* that can replay thousands of randomized disasters against the real consensus code in a way that is exactly reproducible from a single integer, and a *linearizability checker* that mathematically verifies that the answers clients received could have come from a single, honest, non-distributed database.

Contents

1	Orientation: the problem, from zero	3
1.1	Why one computer is not enough	3
1.2	Consensus, from first principles	3
1.3	The majority rule: why three, not two	4
1.4	Linearizability: the correctness target	4
1.5	The project's real thesis	5
2	The map: what is where	6
2.1	Directory structure	6
2.2	The architecture in one picture	7
3	The central design decision: sans-IO	7
4	Raft: the algorithm, and this implementation of it	8
4.1	Terms: logical time	8
4.2	The three roles, and the state machine	9
4.3	Leader election, step by step	10
4.4	Log replication	11
4.5	The commit rule, and the no-op	14
4.6	Persistence and crash safety	15
4.7	Snapshots and log compaction	16
4.8	Membership changes	17
5	Linearizable reads: the ReadIndex protocol	18
6	The KV state machine and exactly-once writes	20
6.1	The exactly-once problem	20

7	The transports	22
7.1	Real TCP	22
7.2	The simulated network	22
8	The deterministic simulator	23
8.1	The virtual clock	23
8.2	One simulation run	24
8.3	The nemesis	24
8.4	The four invariants, checked live	25
8.5	The canary: testing the test	25
8.6	The batch runner	26
9	The linearizability checker	26
9.1	What it is given	26
9.2	The algorithm	26
9.3	The checker's own tests	28
10	Verification: what was actually run for this document	29
10.1	The test suite	29
10.2	The headline sweep	30
10.3	What the numbers do and do not mean	31
11	The outer layers: server, client, CLI	31
11.1	RaftServer	31
11.2	RaftClient	32
11.3	The CLI	32
12	Consolidated honest assessment	32
12.1	What is genuinely strong	33
12.2	Weaknesses, ranked by what would hurt most in an interview	33
12.3	Small things a careful reader will find	33
13	What to take away	34
13.1	The questions this project invites, and the honest answers	35

1 Orientation: the problem, from zero

1.1 Why one computer is not enough

Suppose you store data on one machine. A single hard disk failure, a power cut, or a careless `kill` command and the data is gone or unreachable. The obvious fix is to keep copies on several machines. That word, copies, is where all the difficulty starts.

Jargon: Replication

Keeping more than one copy of the same data on more than one machine, so that the loss of any single machine does not lose the data. Each copy is called a *replica*. The machines are also called *nodes* or *servers*.

Jargon: Node

One participating computer (or, in this project's tests, one process, or even one Python object). In `raft-kv` nodes have short string names: `n1`, `n2`, `n3`.

The naive design is: whenever a write arrives, send it to all three machines. That fails immediately, for reasons worth stating concretely:

- Machine `n3` is briefly down when the write arrives, so it misses it. Its copy is now wrong, and nothing in the system knows.
- Two clients write different values to the same key at the same moment. Machine `n1` happens to process them in one order, `n2` in the other. The two copies now permanently disagree.
- The network splits in half. Machines on each side think the other side died, and both sides keep accepting writes. Now there are two divergent databases claiming to be the same database. This is called *split brain*, and it is the disease that consensus exists to prevent.

Jargon: Split brain

Two or more sets of nodes each believe they are the authoritative cluster, and each accepts writes independently. When the network heals, there is no honest way to merge the two divergent versions of history: acknowledged writes must be thrown away. Any correct consensus algorithm makes split brain structurally impossible rather than merely unlikely.

1.2 Consensus, from first principles

What consensus actually means

Consensus is the problem of getting a group of unreliable machines to agree on one **ordered list of decisions**, such that once a decision is agreed, it is agreed forever, by everyone, and no two machines ever disagree about what decision number 7 was. That is the whole idea. Everything else in Raft is machinery to make that safe under crashes and network failure.

Notice the pivot: the group does not agree on “the value of the key `city`”. It agrees on an ordered *list of commands*:

Position	Command
1	<code>put city = lahore</code>
2	<code>put team = raptors</code>
3	<code>cas team: raptors → dinos</code>
4	<code>delete city</code>

If every machine agrees on that list, and every machine executes the list in order starting from an empty database, then every machine ends up with an identical database, necessarily. Agreement on the data is derived for free from agreement on the list. This trick has a name.

Jargon: Replicated state machine

A design where each node holds (a) an identical ordered log of commands and (b) a deterministic program that executes those commands in order. Because the program is deterministic (same inputs, same order, same outputs, always) and the log is identical, the resulting state is identical on every node. The hard problem is reduced to: keep the logs identical. In this project the log is `raft/log.py`, and the deterministic program is `kv/machine.py`.

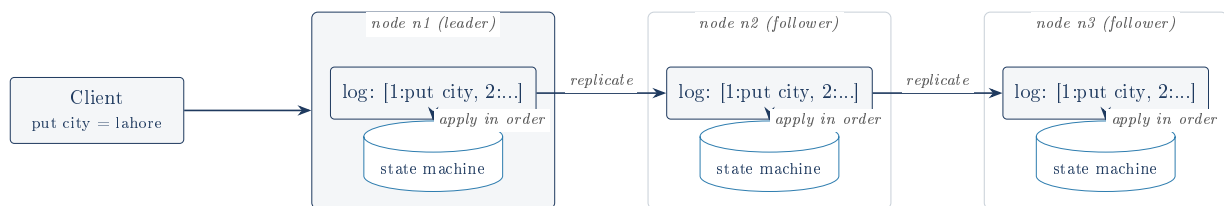


Figure 1: The replicated state machine. Agreement on the ordered log makes agreement on the data automatic, because applying the same commands in the same order to the same starting state must produce the same result.

1.3 The majority rule: why three, not two

The single mechanism that makes split brain impossible is deceptively small.

Quorums, and why they work

Every important decision requires the agreement of a strict majority of the cluster (a *quorum*): 2 out of 3, 3 out of 5. The reason this works is a fact about sets, not about computers: **any two majorities of the same set must share at least one member**. Two majorities of $\{n1, n2, n3\}$ cannot be disjoint, because $2 + 2 > 3$. So if a decision needs a majority, and some node in every majority remembers the previous decision, no second, contradictory decision can slip through unseen. A partition can therefore leave at most *one* side with a majority; the other side is mathematically unable to act.

Jargon: Quorum / majority

More than half the nodes in the current cluster configuration. In this code it is one line, `raft/node.py: len(self.config) // 2 + 1`. For 3 nodes that is 2; for 4 nodes it is 3, which is why even-sized clusters buy you nothing: a 4-node cluster tolerates exactly one failure, the same as a 3-node cluster, while being more expensive and slower.

This is why the project's self-test (section 8.5) attacks precisely this line. Weaken `// 2 + 1` into `// 2` and the majority overlap guarantee evaporates, split brain becomes possible, and the whole edifice falls. It is the load-bearing wall.

1.4 Linearizability: the correctness target

Replication is not the goal. Replication that behaves *as if it were not replicated* is the goal.

Jargon: Linearizability

The strongest single-object consistency guarantee. A system is linearizable if every operation appears to take effect **instantaneously at some single moment between the client sending the request and receiving the response**, and that ordering of moments is consistent with real time. The practical meaning: if your write finished at 10:00:01, then *every* read that starts at 10:00:02, on any node, from any client, must see it. The distributed system is indistinguishable, from the outside, from a single-threaded program holding one copy of the data.

Why linearizability is the interesting property to check

It is a property of the *observable history* (what clients asked, and what they were told), not of the implementation. That makes it checkable from the outside, without trusting the code. If a system's history is linearizable, no client could ever have detected that it was distributed. If it is not, some client saw the seams: a value from the past, a write that vanished, a value that never existed.

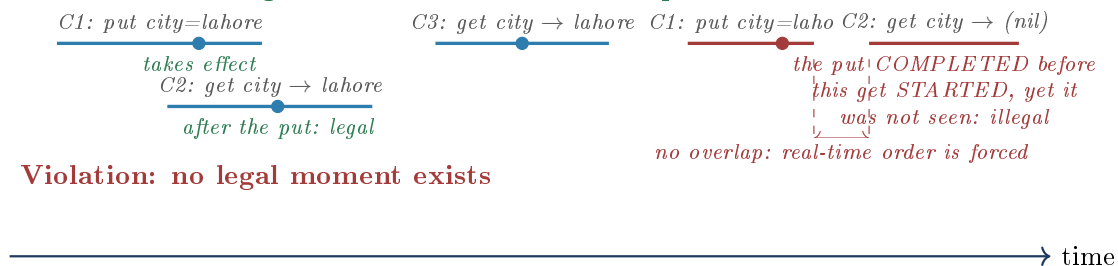
Linearizable: a legal moment exists for each operation

Figure 2: Left: operations overlap in real time, so the checker is free to pick an order that explains every observed result. Right: two operations that do not overlap have their order fixed by real time; if the results contradict that order, no explanation exists and the history is a violation.

1.5 The project's real thesis

The README states the motivation directly: consensus code is easy to write and nearly impossible to trust from example-based tests, because the bugs live in message interleavings you will never hit on a laptop. So the project's central bet, borrowed from the FoundationDB database's testing philosophy, is:

The thesis: make the whole distributed system a pure function of one integer

Every source of randomness in a distributed system (what time it is, how long a message takes, whether a message is lost, when a node crashes, when the network splits, what the client does next) is replaced by draws from a single seeded random number generator. A whole 12-second cluster lifetime under sustained chaos then becomes a *deterministic function of a seed*. That buys three things at once: you can run thousands of universes in seconds because no real time passes; a failure report is one integer rather than a shrug; and the failing schedule replays exactly, forever, under a debugger.

This only works if one condition holds, and the entire architecture is bent around it: **the simulated system must be the real system**. If the simulator tests a copy of the consensus logic, it tests nothing. The mechanism that achieves this is the sans-IO core, section 3.

2 The map: what is where

2.1 Directory structure

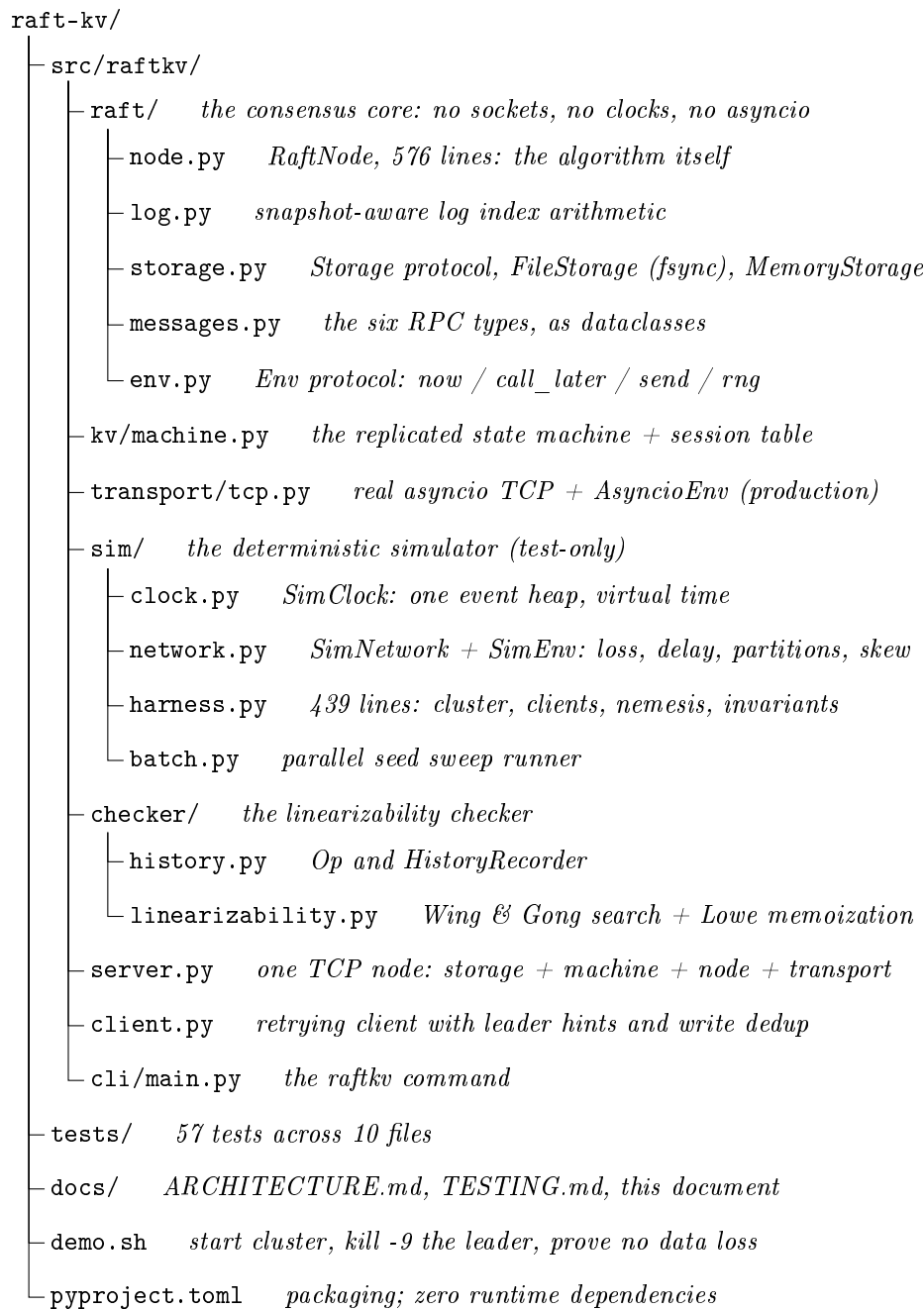


Figure 3: The repository. The single most important boundary in the tree is that `raft/` imports nothing from `sim/` or `transport/`: the arrow of dependency points inward, always.

2.2 The architecture in one picture

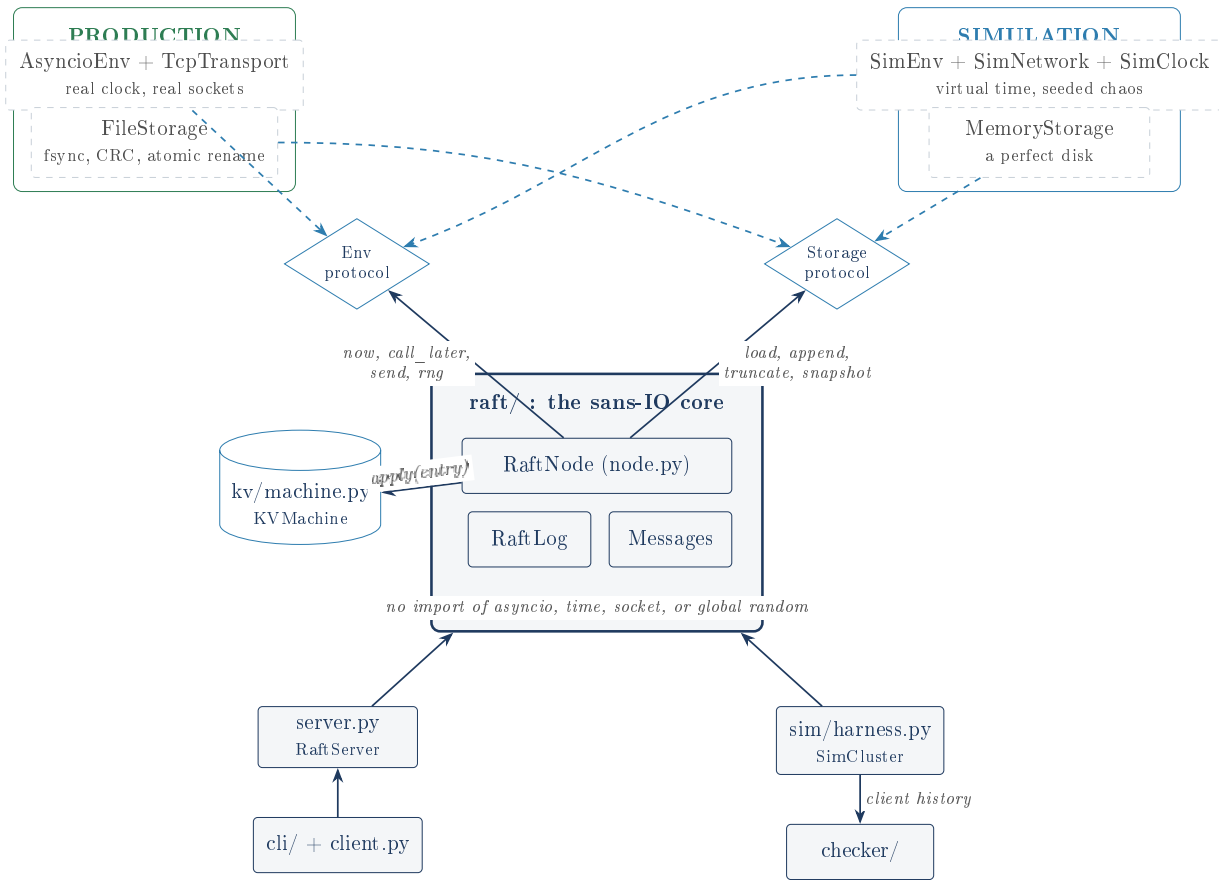


Figure 4: The whole system. `RaftNode` sits in the middle and can only touch the world through two narrow protocols, `Env` and `Storage`. Production plugs real sockets and a real disk into those two sockets; the simulator plugs in a virtual clock and a seeded chaos network. The *same* `RaftNode` object graph runs in both.

3 The central design decision: sans-IO

Jargon: Sans-IO

French for “without IO”. A design style in which a protocol implementation performs no input or output itself. It never opens a socket, never reads a clock, never sleeps. Instead it is a pure-ish state machine: you feed it events (“this message arrived”, “this timer fired”, “the local application wants to propose a command”), and it returns or emits *intentions* (“send this message to n2”, “call me back in 30 milliseconds”). Somebody else, the so-called IO layer, actually performs those intentions.

`RaftNode` reacts to exactly three stimuli and produces effects through exactly two protocols. That is the whole contract, and it fits in a box:

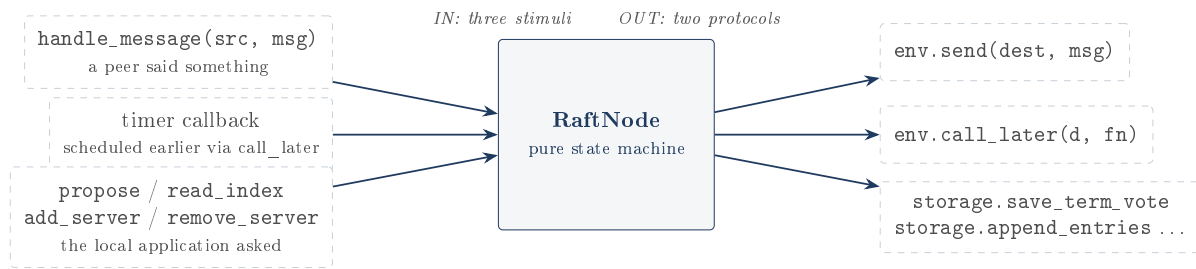


Figure 5: The sans-IO contract of `RaftNode`. Nothing else crosses the boundary.

The `Env` protocol is 6 meaningful lines, in `raft/env.py`:

```
1 class Env(Protocol):
2     rng: random.Random
3
4     def now(self) -> float: ...
5     def call_later(self, delay: float, fn: Callable[[], None]) -> TimerHandle: ...
6     def send(self, dest: str, msg: Message) -> None: ...
```

Listing 1: `raft/env.py`: the entire abstraction that makes the project testable

Why this tiny file is the most important one in the repository

Because `RaftNode` takes its randomness from `env.rng` rather than the global `random` module, and its time from `env.now()` rather than `time.time()`, the simulator can supply a seeded generator and a virtual clock, and the node cannot tell the difference. That is what makes “run 5000 chaos schedules in 90 seconds” an ordinary function call. And crucially, it means every bug the simulator finds is a bug in the code that runs in production, because it *is* the code that runs in production. There is no test double of the algorithm anywhere in this repository.

Note the direction of the dependency arrow. `Env` and `Storage` are declared as `typing.Protocol`, which in Python means structural typing: any object with the right method names satisfies them, with no inheritance and no import. So `raft/` declares what it needs, and `sim/` and `transport/` depend on `raft/`, never the reverse. This is dependency inversion, and it is why `raft/node.py` contains no `if testing:` branch anywhere.

4 Raft: the algorithm, and this implementation of it

Raft is defined by Diego Ongaro and John Ousterhout’s 2014 paper “In Search of an Understandable Consensus Algorithm”, with log compaction and membership changes taken from Ongaro’s Stanford dissertation. This section explains the algorithm from scratch and then shows exactly how `raft/node.py` implements each piece.

4.1 Terms: logical time

Jargon: Term

A number that only ever increases, acting as a logical clock. Each term begins with an election. At most one node may be leader in a given term. Every message carries its sender’s term. The rule is unconditional and lives at the very top of `handle_message`: if you receive any message with a term greater than your own, you immediately adopt that term and become a follower, whatever you were doing. Terms let nodes detect stale information without any assumption about real clocks being synchronized.


```

1 def handle_message(self, src: str, msg: Message) -> None:
2     if msg.term > self.current_term:
3         self._step_down(msg.term)
4     if isinstance(msg, RequestVote):
5         self._on_request_vote(msg)
6     # ... dispatch to the other five handlers

```

Listing 2: raft/node.py: the term rule, applied before dispatch, to every message type

4.2 The three roles, and the state machine

Every node is, at any instant, in exactly one of three roles.

- **Follower:** passive. Answers requests from leaders and candidates. Does nothing on its own except notice silence.
- **Candidate:** campaigning. It got bored of the silence and is asking for votes.
- **Leader:** in charge. It alone accepts client writes, it alone appends to the log, and it sends periodic heartbeats to suppress elections.

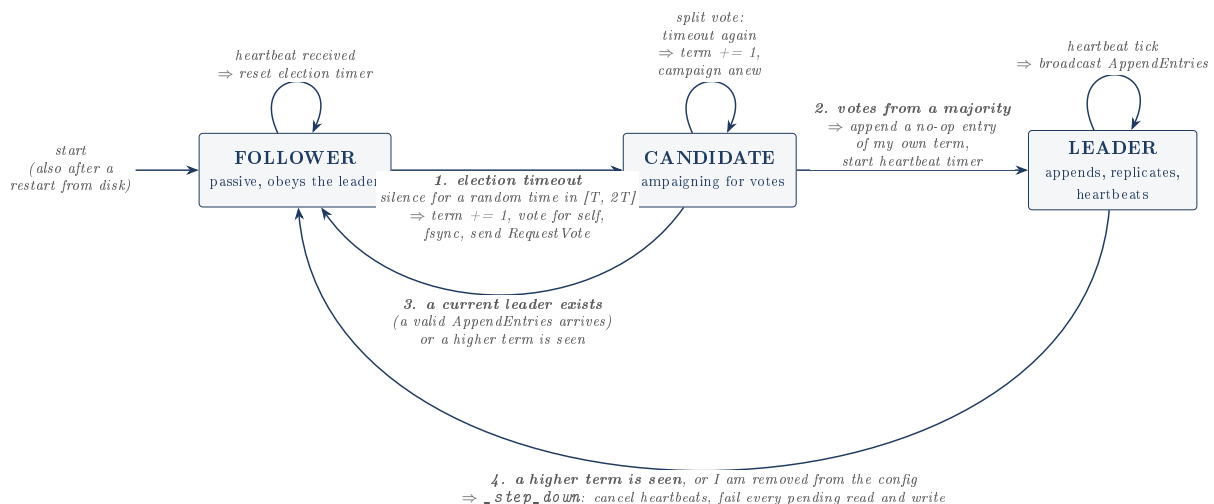


Figure 6: The Raft role state machine as implemented in raft/node.py. Note the asymmetry that makes Raft understandable: there is no follower-to-leader edge and no leader-to-candidate edge. Power is only ever taken through an election, and only ever surrendered by stepping down to follower.

An implementation detail worth knowing: the timer never stops

`_on_election_timeout` re-arms itself *before* checking whether it should act, and returns early if the node is a leader or is not in its own configuration. So the election timer keeps firing forever on every node, even the leader, and even a removed node; it just does nothing. This is harmless and simplifies the code (there is exactly one place that arms the election timer), but it means a reader looking for “where does the leader cancel its election timer” will not find it, because it does not.

4.3 Leader election, step by step

Jargon: Election timeout

How long a follower waits without hearing from a leader before it assumes the leader is dead and starts campaigning. The critical detail is that it is **randomized** per node and per attempt, drawn uniformly from $[T, 2T]$. If it were a fixed constant, all followers would time out at the same instant, all campaign at once, all split the vote, all fail, and repeat forever. Randomization is what makes some node reliably go first and win.

```

1 def _reset_election_timer(self) -> None:
2     if self._election_timer is not None:
3         self._election_timer.cancel()
4     delay = self.env.rng.uniform(self.election_timeout, 2 * self.election_timeout)
5     self._election_timer = self.env.call_later(delay, self._on_election_timeout)

```

Listing 3: raft/node.py: randomized timeout, drawn from the injected rng, never the global one

Election, as implemented in `_start_election` and `_on_request_vote`

```

ON election timeout (follower or candidate, and I am in my own config):
    role ← CANDIDATE
    currentTerm ← currentTerm + 1
    votedFor ← me
    PERSIST (term, votedFor) to storage // before any message leaves
    votes ← {me}
    IF votes ≥ majority THEN become leader // single-node cluster
    send RequestVote(term, me, lastLogIndex, lastLogTerm) to every peer
ON RequestVote(m) received:
    granted ← false
    IF m.term == currentTerm AND votedFor IN {none, m.candidateId}:
        upToDate ← (m.lastLogTerm, m.lastLogIndex) ≥ (myLastLogTerm,
myLastLogIndex)
        IF upToDate:
            granted ← true; votedFor ← m.candidateId
            PERSIST (term, votedFor); reset election timer
            reply RequestVoteReply(currentTerm, me, granted)
ON RequestVoteReply(m), if I am still a CANDIDATE in term m.term:
    IF m.voteGranted AND m.voterId in my config: votes ← votes ∪ {m.voterId}
    IF |votes| ≥ majority: become leader

```

Two subtleties in that pseudocode carry enormous weight.

Persist the vote before replying, or lose safety

`_persist_term_vote()` is called *before* the reply is sent, both when campaigning and when granting a vote. If a node granted a vote, crashed, recovered with no memory of it, and granted a second vote for the same term to a different candidate, two candidates could each collect a majority in one term, and there would be two leaders. Durability of `votedFor` is not bookkeeping; it is what makes “at most one leader per term” true across crashes.

The up-to-date check: elections are a filter, not a lottery

A voter refuses any candidate whose log is less up to date than its own, where “more up to date” means: higher last term wins; if the last terms tie, the longer log wins. In Python that comparison is literally a tuple comparison, `(msg.last_log_term, msg.last_log_index) >= (self.log.last_term, self.log.last_index)`. This is the paper’s section 5.4.1, and it is

what makes the *Leader Completeness* property true: since a winner needs a majority, and any majority overlaps the majority that stored any committed entry, and every voter refuses anyone missing entries it holds, a node missing a committed entry can never win. Committed data therefore survives every leader change, automatically, with no data transfer during the election itself.

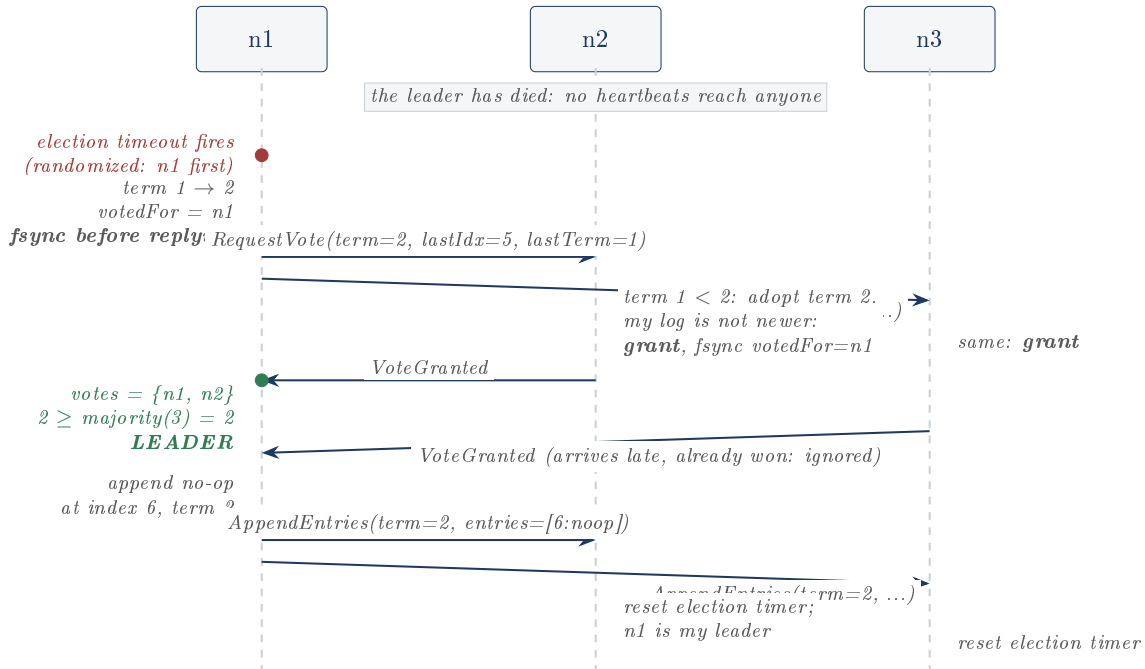


Figure 7: One election, end to end. n1's randomized timer fires first, so it campaigns alone and wins immediately; n3's vote arrives after the fact and is ignored. The new leader's first act is to append a no-op entry, for the reason explained in section 4.5.

4.4 Log replication

Jargon: Log entry

One record in the ordered list. In `raft/messages.py` it is a frozen dataclass with four fields: **index** (its 1-based position, never reused), **term** (the term of the leader that created it), **kind**, and **data**. There are exactly three kinds in this codebase: "command" (a KV operation to run), "config" (a cluster membership change), and "noop" (an empty entry a new leader appends to itself).

Jargon: Commit / committed

An entry is *committed* once the leader has stored it on a majority of the cluster. Committed means permanent: it will be present in every future leader's log and will eventually be applied on every node. Uncommitted entries can still be thrown away. `commit_index` is the highest index known committed; `last_applied` is the highest index actually executed against the state machine. The gap between them is entries that are decided but not yet run.

The leader tracks two numbers per follower, and understanding the difference is most of understanding replication:

Field	Meaning	Nature
<code>next_index[p]</code>	the next index I will <i>try</i> to send p	optimistic guess
<code>match_index[p]</code>	the highest index I <i>know</i> p has stored	proven fact

On becoming leader, `next_index` is set optimistically to `last_index + 1` for everyone and `match_index` to 0 for everyone: “I assume you are caught up, and I have proof of nothing.” Reality then corrects the guess.

AppendEntries on the follower (`_on_append_entries`), the consistency check

```

ON AppendEntries(m):
  IF m.term < currentTerm:  reply(success=false)  // stale leader, ignore
  IF role != FOLLOWER: step down  // a valid leader exists for this term
  leaderHint ← m.leaderId; reset election timer
  // THE LOG MATCHING CHECK: do we agree on the entry just before these?
  IF term_at(m.prevLogIndex) != m.prevLogTerm:
    // We disagree. Tell the leader WHERE, so it can back up fast.
    IF we have no entry there at all:  conflict ← myLastIndex + 1
    ELSE: conflict ← first index of the conflicting term
    reply(success=false, conflictIndex=conflict); RETURN
  // We agree up to prevLogIndex. Merge the new entries.
  FOR each entry e in m.entries:
    IF we have nothing at e.index:  append it
    ELSE IF our term at e.index != e.term:
      TRUNCATE our log from e.index onward  // ours was wrong; leader wins
      append e
    // ELSE: identical entry already present, skip
  persist appended entries (fsync); refresh config if a config entry moved
  IF m.leaderCommit > commitIndex:  commitIndex ← min(m.leaderCommit, match)
  reply(success=true, matchIndex=m.prevLogIndex + len(m.entries), seq=m.seq)

```

The Log Matching Property, and why one check suffices

The check “do we agree on the single entry immediately before the ones I am sending” looks far too weak to guarantee that two whole logs agree. It is not, because of an induction: entries are only ever appended by a leader, indexes are never reused, and only one leader exists per term. So if two logs contain an entry with the same index *and* the same term, that entry was created by the same leader in the same term, and therefore all entries before it are also identical. Agreement on one entry proves agreement on the entire prefix. That is why `AppendEntries` carries `prev_log_index` and `prev_log_term` at all, and why the follower’s reply is trustworthy.

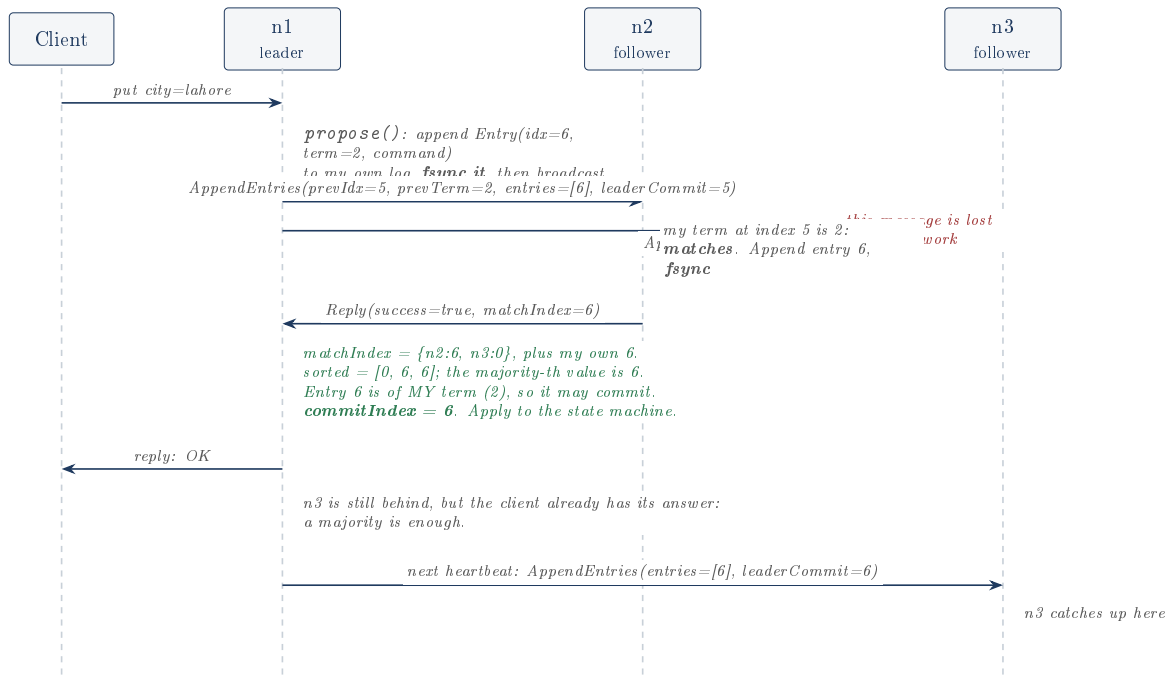


Figure 8: The replication path for one write. The client is answered as soon as a *majority* has the entry stored durably; the straggler catches up on the next heartbeat. Losing n3’s message costs nothing, because `AppendEntries` is idempotent and heartbeats retry forever.

4.4.1 Fast backoff on conflict

When a follower rejects an `AppendEntries`, the naive repair is for the leader to decrement `next_index` by one and try again, costing one full round trip per missing entry. A follower 500 entries behind then needs 500 round trips. Raft’s optimization, implemented here, is for the follower to return a *conflict hint*: the first index of the conflicting term. The leader jumps straight there.

```

1  our_prev_term = self.log.term_at(prev_index)
2  if our_prev_term is None or our_prev_term != prev_term:
3      if our_prev_term is None:
4          conflict = self.log.last_index + 1          # our log is simply too short
5      else:
6          conflict = prev_index                      # walk back over the whole bad term
7          while conflict > self.log.snapshot_index + 1 and \
8                self.log.term_at(conflict - 1) == our_prev_term:
9              conflict -= 1
10     self.env.send(msg.leader_id, AppendEntriesReply(
11         term=self.current_term, follower_id=self.id, success=False,
12         match_index=0, conflict_index=conflict, seq=msg.seq))
13     return

```

Listing 4: raft/node.py: the follower computes the hint by scanning back over its own conflicting term

```

1  else:
2      # Fast backoff using the follower's conflict hint.
3      self.next_index[p] = max(1, min(self.next_index[p] - 1, msg.conflict_index)
4                                  if msg.conflict_index else self.next_index[p] - 1)
5      self._send_append(p)

```

Listing 5: raft/node.py: and the leader consumes it, but never trusts it upward

The `min(next_index - 1, conflict_index)` is a defensive detail worth noticing in an interview: the hint can only ever move `next_index` *down*, never up. A hint from a confused or reordered reply can therefore cost an extra round trip but can never skip entries, which would be a correctness bug rather than a performance one. The `max(1, ...)` floors it at the first real index.

4.5 The commit rule, and the no-op

This is the single subtlest rule in Raft, and the README singles it out as one of the two places “where a plausible shortcut is a real linearizability bug”.

```

1 def _maybe_advance_commit(self) -> None:
2     if self.role is not Role.LEADER:
3         return
4     matches = sorted(
5         (self.match_index[n] if n != self.id else self.log.last_index)
6         for n in self.config
7     )
8     if not matches:
9         return
10    candidate = matches[len(matches) - self._majority()]
11    if candidate > self.commit_index and self.log.term_at(candidate) == self.current_term:
12        self._set_commit_index(candidate)

```

Listing 6: raft/node.py: `_maybe_advance_commit`, all seven lines of it

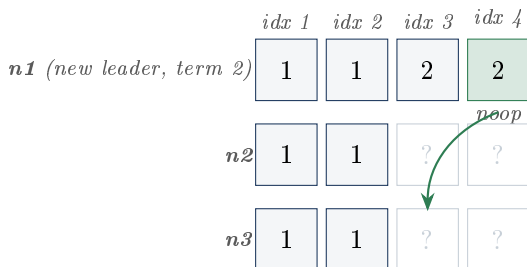
The first half is a neat trick: sort every node’s match index (counting the leader’s own last index for itself) and take the element at position `len - majority`. For 3 nodes with matches [0, 6, 6], that is index $3 - 2 = 1$, giving 6: the highest index that a majority has reached. It is the median, generalized.

The second half is the subtle part: `self.log.term_at(candidate) == self.current_term`. A leader may **not** commit an entry from an earlier term merely because it is now replicated on a majority.

Why counting replicas of an old entry is unsafe (the paper’s Figure 8)

Suppose an entry from term 2 sits on a majority of nodes but was never committed by term 2’s leader (which crashed before it could count). A new leader in term 4 sees it on a majority and commits it. But a *different* node, whose log happens to be more up to date on other grounds, can still win a later election, and its log does not contain that entry. It would then overwrite it. An entry that was reported committed would vanish. Replication count alone is therefore not proof of permanence for old entries.

The fix, and this implementation’s version of it: a leader may only directly commit entries **from its own term**. To make progress on the backlog, a fresh leader immediately appends a **noop** entry of its own term (`_become_leader`, line 212). When that no-op commits by the ordinary majority rule, every entry before it commits *transitively*, by the Log Matching Property, safely. The no-op is not a nicety; without it a leader that receives no writes could never commit its predecessor’s backlog at all.



FORBIDDEN: n1 may not commit index 3 (term 1... here term 2 but created by the PREVIOUS leader) just because a majority stores it. Old-term entries are not committable by replica count.

ALLOWED: once the term-2 noop at index 4 reaches a majority, index 4 commits by the ordinary rule, and indexes 1 to 3 commit transitively with it, because any log holding entry 4 must hold its whole prefix.

Figure 9: The section 5.4.2 commit restriction. The no-op entry is the vehicle that safely drags the previous leader’s backlog into the committed region.

The test `test_commit_only_with_current_term_entry` pins exactly this: it force-installs a leader with two term-1 entries, asserts `commit_index == 0` immediately (nothing commits before replica-

tion), runs, and asserts `commit_index == 3`: the no-op at index 3 committed and pulled indexes 1 and 2 with it.

4.6 Persistence and crash safety

Jargon: fsync

An operating system call that does not return until the data you wrote has actually reached durable storage. An ordinary `write()` only copies data into a kernel buffer in memory; if the power fails a moment later, that data never existed. Without `fsync`, all of Raft's durability reasoning is fiction.

`FileStorage` keeps three files per node and uses two different, carefully chosen durability techniques:

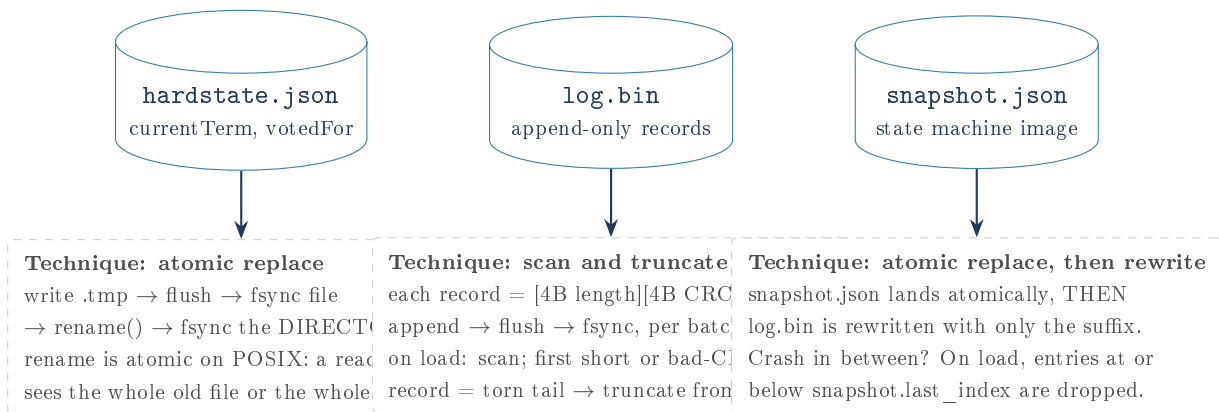


Figure 10: The three on-disk structures and their recovery strategies. Each is either atomic-rename recoverable or scan-and-truncate recoverable, which is the project's stated one-line discipline for crash safety.

Why a torn tail is always safe to discard

A crash mid-append can leave half a record on disk. Recovery finds the bad CRC32 checksum and truncates from that point. This throws away data, which sounds alarming, and is provably fine: Raft never acknowledges anything before `fsync` returns, so a record that was still being written was, by definition, never acknowledged to anybody. Nobody was told it existed, so nobody can be surprised that it does not. Discipline (never acknowledge before `fsync`) is what makes a destructive recovery correct.

The scan is genuinely as small as the README claims. This is all of it:

```

1 while pos + _REC_HEADER.size <= len(raw):
2     length, crc = _REC_HEADER.unpack_from(raw, pos)
3     start = pos + _REC_HEADER.size
4     payload = raw[start:start + length]
5     if len(payload) < length or zlib.crc32(payload) != crc:
6         break # torn tail from a crash mid-write
7     entries.append(Entry.from_dict(json.loads(payload)))
8     pos = start + length
9     good_end = pos
10 if good_end < len(raw):
11     # Discard the torn tail so future appends start from a clean point.
12     self._log_fh.close()
13     with open(self._logf, "r+b") as f:
14         f.truncate(good_end)
15         f.flush()

```

```

16         os.fsync(f.fileno())
17         self._log_fh = open(self._logf, "ab")

```

Listing 7: raft/storage.py: torn-tail recovery, the whole mechanism

Note the detail in `_atomic_write`: after `os.rename`, the code opens the containing *directory* and fsyncs that too. Renaming is atomic, but the directory entry recording the rename is itself buffered; without the directory fsync, a power cut can leave the file at its old name. This is a classic mistake, and it is not made here.

Two tests drive these paths directly rather than by inspection:

- `test_crash_mid_write_discards_torn_tail`
writes a valid record header promising 500 bytes and then supplies 11, then asserts both that the surviving entries are exactly [1, 2, 3] and that the file was physically truncated back to its previous size.
- `test_corrupt_record_stops_scan`
flips a single bit inside the last record's payload and asserts the CRC rejects it.

4.7 Snapshots and log compaction

An append-only log grows forever. Compaction replaces a prefix of the log with a single image of the state that prefix produced.

Jargon: Snapshot

A serialized copy of the entire state machine at a particular log index, plus the metadata needed to keep the log arithmetic consistent: `last_included_index`, `last_included_term`, and the cluster config as of that point. Here the payload is `KVMachine.snapshot()`: a JSON blob of the data dictionary *and* the session table.

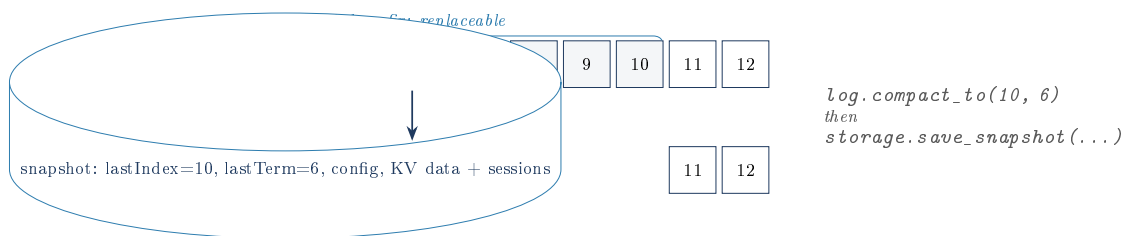


Figure 11: Compaction. RaftLog keeps `snapshot_index` as an offset, so index arithmetic on the surviving entries stays correct: `entry(i)` resolves to `entries[i - snapshot_index - 1]`.

The trigger is in `_maybe_snapshot`, called after every apply:

```

1 if self.last_applied - self.log.snapshot_index < self.snapshot_threshold:
2     return

```

Above the threshold (1000 entries in production, 40 in the simulator, 10 in the snapshot tests) it takes an image and compacts.

The complication compaction creates is that a leader may no longer possess the entries a lagging follower needs, because it deleted them. `_send_append` detects this in its first two lines and switches protocol:

```

1 def _send_append(self, p: str) -> None:
2     ni = self.next_index[p]
3     if ni <= self.log.snapshot_index:
4         snap = self._load_snapshot()
5         self.env.send(p, InstallSnapshot(

```



```

6         term=self.current_term, leader_id=self.id,
7         last_included_index=snap.last_index,
8         last_included_term=snap.last_term,
9         config=tuple(snap.config), data=snap.data))
10    return
11    prev_term = self.log.term_at(ni - 1)
12    assert prev_term is not None
13    entries = tuple(self.log.slice_from(ni))
14    self.env.send(p, AppendEntries(...))

```

Listing 8: raft/node.py: the fork between replication and snapshot transfer

On the receiving side, `_on_install_snapshot` contains a nice piece of care. If the follower’s own log already contains an entry at `last_included_index` *with the matching term*, the snapshot is merely a prefix of what it already has, so it compacts and keeps its suffix. Only if the terms disagree does it discard its log wholesale:

```

1  if self.log.term_at(msg.last_included_index) == msg.last_included_term:
2      # Our log already extends past the snapshot; keep the suffix.
3      self.log.compact_to(msg.last_included_index, msg.last_included_term)
4  else:
5      self.log = RaftLog(msg.last_included_index, msg.last_included_term)

```

Listing 9: raft/node.py: keep the suffix when the snapshot is only a prefix of what we hold

Snapshots are read from disk and shipped whole, on every send

`_load_snapshot()` is literally `return self.storage.load().snapshot`. For `FileStorage` that re-reads and re-parses `snapshot.json` from disk **every single time** a snapshot is sent, and `load()` also re-reads and re-parses the entire `log.bin` while it is there, discarding the result. It is then sent as one message with no chunking, so a large state machine means a single enormous frame (bounded only by `MAX_FRAME`, 64 MB). At three nodes and a demo-sized dataset this is invisible; at any real scale it is the first thing that would fall over. The README lists the chunked transfer and cached handle as known future work, which is honest, but the severity is easy to understate: the cost here is not just “re-reads the snapshot”, it is “re-reads and re-parses the entire log too”.

4.8 Membership changes

Changing the cluster from $\{n1, n2, n3\}$ to $\{n1, n2, n3, n4\}$ is dangerous for a reason that is easy to miss: during the switch, different nodes believe in different configurations, and if the old and new majorities do not overlap, two leaders can be elected simultaneously without either breaking any rule.

Why single-server changes are safe without a two-phase protocol

Add or remove exactly **one** server per config entry. Any two configurations differing by one server must share a common majority: a majority of $\{n1, n2, n3\}$ is 2, a majority of $\{n1, n2, n3, n4\}$ is 3, and any 2-of-3 and any 3-of-4 drawn from these overlapping sets must intersect. So the disjoint quorums that cause dual leadership cannot form, and no special “joint” state is needed. Ongaro’s dissertation’s alternative, *joint consensus*, allows arbitrary jumps but costs a two-phase protocol and a state where both old and new majorities must agree. `docs/ARCHITECTURE.md` states the tradeoff and the choice explicitly.

The implementation is a plain log entry of kind `config` plus exactly two extra rules:

1. **One at a time.** `_config_change_pending()` scans the uncommitted tail for any config entry and raises `ConfigChangeInProgress` if one is there.

2. **Effective on append, not on commit.** `_refresh_config()` runs the moment a config entry is appended or truncated. This is counter-intuitive (acting on data that might still be rolled back) but is the dissertation’s rule, and it is necessary: a node must count itself out of a cluster it has been removed from immediately, or it can disrupt the very commit that removes it.

Leader self-removal is handled at the end of `_apply_committed`: the leader keeps leading until the config entry that removes it *commits*, and only then steps down, so it can shepherd its own removal to completion.

```
1 if entry.kind == "config" and self.role is Role.LEADER and \
2     self.id not in tuple(entry.data["nodes"]):
3     self._step_down(self.current_term)
```

Listing 10: raft/node.py: a leader that has been removed from the config steps down, but only on apply

A joining node is started with an empty `initial_config`. Because `_on_election_timeout` returns early when `self.id not in self.config`, and an empty config contains nothing, a joiner never campaigns. It sits silent until the leader’s `AppendEntries` or `InstallSnapshot` teaches it the cluster it belongs to. This is elegant: one guard clause implements “joining nodes must not disrupt the cluster”.

A real dead end: `_config_from`’s `entries_last_index` filter

`_config_from(entries_last_index)` filters entries above a bound, but the only caller that passes a bound is `_maybe_snapshot`, and it passes `self.last_applied`, which by construction is a prefix of the log at that moment. The other caller, `_latest_config`, passes `None`. The filter is therefore correct but currently near-vacuous. Not a bug; worth knowing before an interviewer asks what it defends against.

An inconsistency between two nearly identical lines

`_become_leader` initializes `next_index` to `self.log.last_index + 1`. `_change_config` initializes newly appearing peers to `self.log.last_index` (no + 1), while `_refresh_config` uses `last_index + 1` again. The + 1 is the paper’s value. The odd one out in `_change_config` is off by one and means the leader will send the new server one entry it need not have sent (the config entry itself, which it does not have anyway, so the follower rejects and the conflict hint repairs it in one extra round trip). It is self-correcting and therefore invisible in the tests, but the three lines should agree, and an interviewer reading these three functions side by side will notice.

5 Linearizable reads: the ReadIndex protocol

Writes go through the log, so they are safe by construction. Reads are the trap.

Why reading from the leader’s memory is wrong

“I am the leader, so I will just answer from my local copy” is unsafe, because **a leader cannot know it is still the leader**. Suppose `n1` is partitioned away from `n2` and `n3`. Those two elect `n3` in a new term and accept writes. `n1` has heard nothing, so it still believes it is leader, its local data looks perfectly fine to it, and it will happily serve a value that is now minutes out of date. Nothing about its own state betrays it. Leadership is not a local fact.

The fix is to make the leader *prove* its currency to a majority before answering, which is the ReadIndex protocol:

ReadIndex (read_index + _check_pending_reads)

```

ON read_index(callback):
  IF not leader: callback(None); RETURN // caller must redirect
  // 1. A fresh leader must first commit an entry of its OWN term.
  readIdx ← commitIndex IF an entry of my term has committed ELSE none
  // 2. Record the current heartbeat round number and broadcast it.
  hbSeq ← hbSeq + 1
  pendingReads.append(PendingRead(seq=hbSeq, readIndex=readIdx, cb=callback))
  send AppendEntries(seq=hbSeq) to every peer
ON any AppendEntriesReply(seq=s) from peer p: ackSeq[p] ← max(ackSeq[p], s)
CHECK pending reads (after every reply, every commit, every broadcast):
  FOR each pending read r with r.readIndex != none:
    acks ← count of nodes n where n == me OR ackSeq[n] ≥ r.seq
    IF acks ≥ majority AND lastApplied ≥ r.readIndex:
      remove r; r.cb(r.readIndex) // NOW reading local state is safe
ON step down (any cause): FOR each pending read r: r.cb(none) // fail them all

```

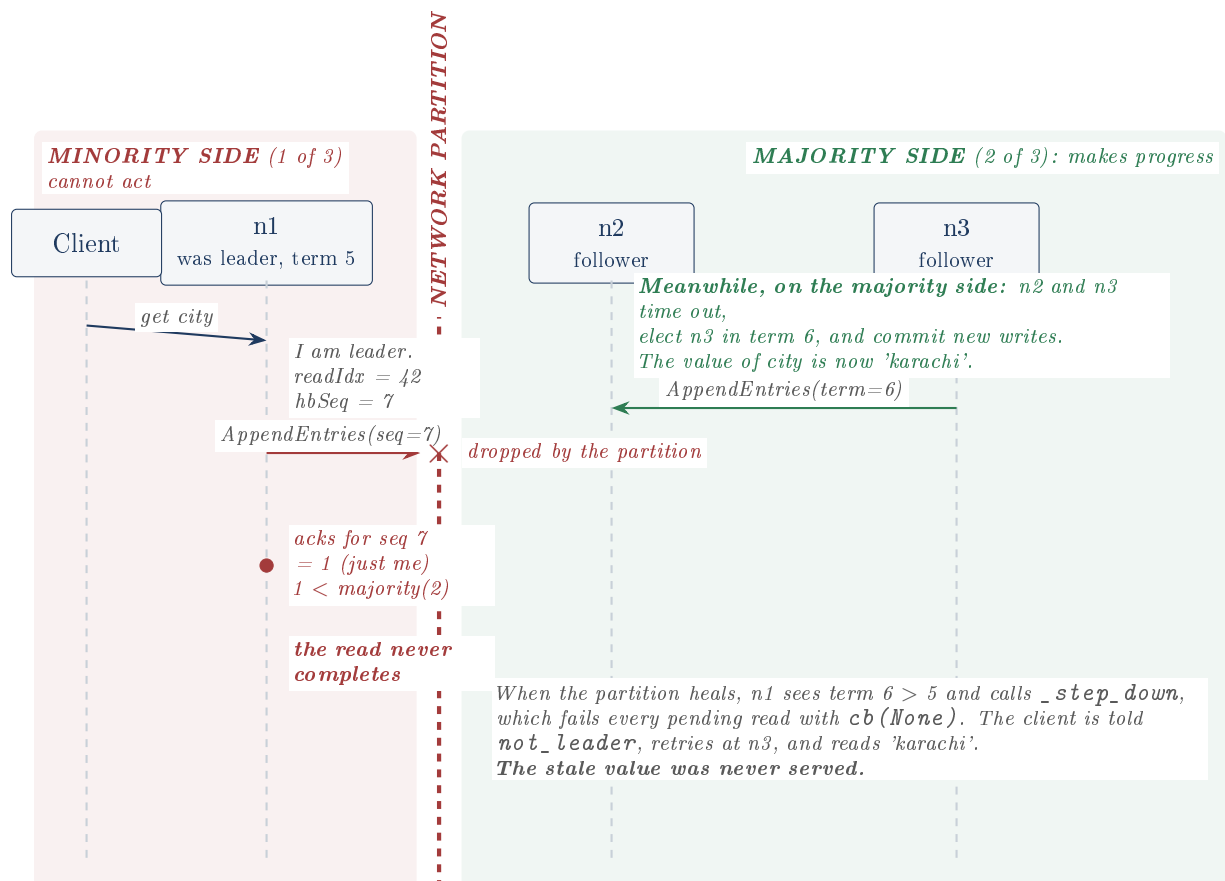


Figure 12: A network partition, and why ReadIndex is safe. The deposed leader is not merely unlikely to answer; it is structurally unable to, because the acks it needs cannot cross the partition. Its own state gives it no clue that anything is wrong, which is exactly why local state must never be trusted.

The second precondition: a fresh leader must commit its own term first

The README calls this out as the bug the chaos harness found. A newly elected leader's `commit_index` is a stale local variable: it reflects what *it* knew before the election, and *may be lower than what is actually committed cluster-wide*, because commit index is not transferred during elections. Capturing `readIndex = commitIndex` at that moment could therefore serve

a read that misses a committed write, which is a real linearizability violation. The implementation handles this without stalling: a read that arrives too early is registered with `read_index = None`, and `_set_commit_index` assigns it the moment the first entry of the leader's own term (the no-op) commits:

```
1 if self.role is Role.LEADER and self._term_commit_index is None and \
2     self.log.term_at(index) == self.current_term:
3     self._term_commit_index = index
4     for r in self._pending_reads:
5         if r.read_index is None:
6             r.read_index = self.commit_index
```

Listing 11: raft/node.py: pending reads get their index the instant the term's first entry commits

`docs/ARCHITECTURE.md` also documents the alternative that was rejected: *leader leases*, which let a leader serve reads for `election_timeout - clock_drift` after a heartbeat quorum, saving the per-read round trip. The stated reason for refusing it is exact and worth repeating in an interview: leases buy latency at the price of a safety argument that depends on **bounded clock drift between physical machines**. In a project whose entire thesis is testable correctness, importing an untestable assumption about hardware clocks would be self-defeating. `ReadIndex` costs one heartbeat round and assumes nothing about clocks.

A pleasing efficiency: the heartbeat round is shared. Concurrent reads piggyback on `_hb_seq` windows, and `_ack_seq[p]` is monotone (`max(...)`), so an ack for round 9 automatically satisfies a read waiting on round 7.

6 The KV state machine and exactly-once writes

`kv/machine.py` is 72 lines and is the deterministic program from the replicated state machine definition. Four operations:

Op	Effect	Result dict
put	<code>data[key] = value</code>	<code>{ok: True}</code>
delete	remove key	<code>{ok: True, existed: bool}</code>
cas	set only if current value == expected	<code>{ok, swapped: bool, value: old}</code>
get	never in the log; via <code>ReadIndex</code>	<code>{value}</code>

Jargon: Compare-and-swap (cas)

“Set **key** to **new**, but only if it currently holds **expected**.” It is the primitive that makes safe concurrent updates possible without locks: a client can read a value, compute a new one, and cas it, and the cas fails harmlessly if anybody else changed the key in between. In this codebase, a cas with `expected=None` means “only if the key is absent”, and a cas with `value=None` means “delete if it currently equals expected”.

6.1 The exactly-once problem

Why retries silently corrupt data, and why the fix must live in the state machine

A client sends `cas team: raptors → dinos`. The leader commits it, then dies before the reply is sent. The client times out and retries against the new leader. The new leader has no idea this is a retry: it applies the cas a second time. Now the value is **raptors**... no: the value is already **dinos**, so the second cas *fails*, and the client is told its operation failed when in fact

it succeeded. For a non-idempotent operation like “increment”, a retry would double-apply outright.

The fix: every write carries `(client_id, seq)`. The state machine keeps, per client, the last seq it applied and the *result it returned*. A duplicate returns the cached result without re-executing.

The part that is easy to get wrong: this table must live **inside the replicated state**, replicated through the log and included in snapshots. If it were a leader-local cache, failover would silently downgrade the system from exactly-once to at-least-once, precisely when it matters most, because the new leader would have no memory of what the old one had already done.

```

1 def apply(self, entry: Entry) -> Result:
2     cmd = entry.data
3     client, seq = cmd.get("client"), cmd.get("seq")
4     if client is not None and seq is not None:
5         cached = self.sessions.get(client)
6         if cached is not None and seq <= cached[0]:
7             return cached[1] # duplicate: original result, no re-execution
8     result = self._execute(cmd)
9     if client is not None and seq is not None:
10        self.sessions[client] = (seq, result)
11    return result

```

Listing 12: kv/machine.py: the entire dedup mechanism, at the top of `apply()`

And the snapshot carries it, which is the point:

```

1 def snapshot(self) -> str:
2     return json.dumps({"data": self.data, "sessions": self.sessions})

```

Listing 13: kv/machine.py: sessions ride inside the snapshot, not beside it

`test_exactly_once_across_leader_failover` drives exactly the scenario above: commit a cas, crash the leader before the client hears back, retry the identical `(client, seq)` on the new leader, and assert the retry returns `swapped: True` (the *cached original* outcome, not the correct-but-wrong answer a fresh evaluation would give) and that the value was applied once.

The dedup rule is `seq <= cached[0]`, which is stricter than it looks

Because the check is “less than or equal to the highest seq I have seen”, a write that arrives with an *older* seq than one already applied is silently discarded as a duplicate, returning the newer operation’s cached result. That is correct only if each client’s seqs are strictly monotonic. The README describes this biting the project’s own test harness: a helper issued writes with random seq numbers, they were correctly discarded, and two unrelated tests failed mysteriously. The lesson was baked into `client.py` (`self._seq = itertools.count(1)`) and into the simulator’s client (which uses the globally monotonic `op.op_id` as its seq). It is worth being clear about the shape of the contract: **the correctness of the dedup table is a constraint imposed on every client, enforced nowhere in the server**. A client library with a bug here corrupts its own results and nothing detects it.

Sessions never expire, and the leak is unbounded in the wrong dimension

`self.sessions[client] = (seq, result)` is never removed. The README lists this. What the README does not spell out is that the table is not just a memory leak in RAM: it is part of every snapshot, so it is written to disk, shipped over the network in full on every `InstallSnapshot`, and grows with the number of *distinct client ids ever seen*. `RaftClient` generates a fresh random `cli-<uuid>` per instance by default, and the CLI constructs a new

client per command, so **every single raftkv put from the command line permanently adds a row**. The dissertation's fix (a session TTL agreed through the log) is correctly identified as the prerequisite for calling this production-ready.

7 The transports

7.1 Real TCP

Jargon: Length-prefixed framing

TCP is a stream of bytes with no message boundaries: send two 100-byte messages and the receiver may get 200 bytes at once, or 3 then 197. So every message is prefixed with its length (here 4 bytes, big-endian), and the reader reads exactly 4 bytes, decodes the length, then reads exactly that many. This codebase uses `readexactly()` for both, which is the correct primitive.

Design notes worth defending:

- **One port for peer and client traffic**, distinguished by a `kind` field in the frame ("hello", "raft", or anything else meaning a client request).
- **Peer sends are deliberately lossy**. Each peer has a bounded `asyncio.Queue(maxsize=256)` drained by a dedicated writer task that reconnects on failure. If the queue is full, **except** `asyncio.QueueFull: pass`, and the message is dropped without comment. This looks like a bug and is a design decision: Raft already assumes a lossy network and retransmits on the next heartbeat, so dropping is strictly better than growing an unbounded queue of stale messages that will be obsolete by the time they are sent.

The Python 3.12 shutdown deadlock (a real bug, worth telling)

Python 3.12 changed `asyncio.Server.wait_closed()` to also wait for in-flight connection *handlers*, not just the listening socket. Every client connection in this design sits parked in `readexactly`, waiting forever for a next request that will never come. So `wait_closed()` never returned: the TCP integration test hung for minutes at approximately zero CPU, which is the signature of a deadlock rather than a slow loop. The fix is the `self._conns` set: track every accepted `StreamWriter` and close them all *before* awaiting `wait_closed()`, so the parked handlers get an EOF, return, and let the server finish closing.

7.2 The simulated network

`SimNetwork` implements the same send/deliver contract over the virtual clock, with four seeded faults. The whole of `send` is:

```

1 def send(self, src: str, dest: str, msg: Message) -> None:
2     self.messages_sent += 1
3     if self.rng.random() < self.drop_prob:           # 1. loss
4         self.messages_dropped += 1
5         return
6     copies = 2 if self.rng.random() < self.dup_prob else 1    # 2. duplication
7     for _ in range(copies):
8         delay = self.rng.uniform(self.min_delay, self.max_delay) # 3. reordering, via delay
9         self.clock.call_later(delay, lambda: self._deliver(src, dest, msg))
10
11 def _deliver(self, src: str, dest: str, msg: Message) -> None:
12     handler = self._handlers.get(dest)
13     if handler is None or self.is_partitioned(src, dest):    # 4. partition, at DELIVERY time

```

```

14         self.messages_dropped += 1
15         return
16         handler(src, msg)

```

Listing 14: sim/network.py: every fault, in nine lines

Why partitions are checked at delivery, not at send

Because that is what a real network does. If a partition forms while a message is in flight, the message dies in transit; the TCP connection carrying it resets. Checking at send time would model a network that politely refuses to accept doomed messages, which no network does, and would miss the entire class of bugs where a message crosses a boundary that closed behind it. Independent random delay per copy also means duplicates arrive out of order, for free.

SimEnv adds two things the real AsyncioEnv does not need:

- **timer_scale**, multiplying every scheduled delay, which models clock skew. A node with scale 0.5 fires its election timers twice as fast as everyone else; the nemesis rolls scales in [0.5, 1.8].
- **kill() and the epoch guard**. A simulated crash must invalidate every callback the dead node scheduled, including any already sitting on the shared heap. Every callback is wrapped in a closure that compares the epoch it was created in against the current one and silently does nothing if they differ. This is how “the process died” is modelled without unwinding the heap.

```

1 def call_later(self, delay: float, fn: Callable[[], None]) -> SimTimer:
2     epoch = self._epoch
3     def guarded() -> None:
4         if epoch == self._epoch:
5             fn()
6     t = self.clock.call_later(delay * self.timer_scale, guarded)
7     ...
8 def kill(self) -> None:
9     """Invalidate all outstanding timers (simulated crash)."""
10    self._epoch += 1
11    ...

```

Listing 15: sim/network.py: the epoch guard, i.e. how a node stops existing

8 The deterministic simulator

8.1 The virtual clock

SimClock is 57 lines and is the beating heart of the determinism claim. One binary heap of events, ordered by (**when**, **seq**). **step()** pops the earliest, jumps **now** to its timestamp, and runs it.

Why the tie-break on a sequence number is load-bearing

Two events scheduled for the same virtual instant must still run in a defined order, or determinism dies. `SimTimer._lt_` compares (`self.when`, `self.seq`), and `seq` is a monotonically increasing counter assigned at scheduling time. Without it, Python’s heap would break ties by whatever happened to be in the array, and the same seed could produce two different runs. The README describes exactly this class of bug: two accidental uses of ambient ordering, caught by a test that runs a seed twice and compares full history fingerprints.

And because time is a variable rather than a thing you wait for, “12 virtual seconds of a 3-node cluster” costs roughly 0.1 real seconds. There is no `sleep` anywhere in the simulator. That single fact is what converts “test the distributed system” from an overnight job into a function call.

8.2 One simulation run

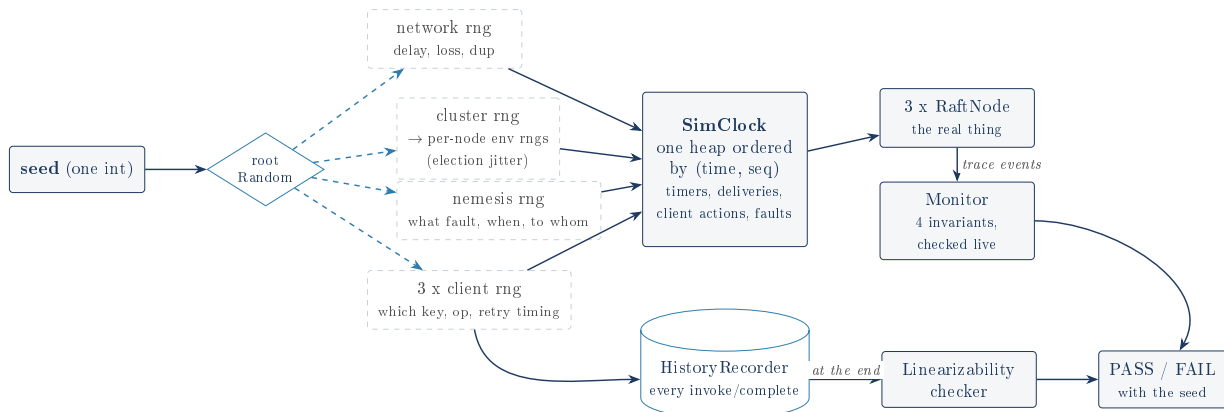


Figure 13: One seeded run. The seed fans out into six independent generators, all events funnel through one ordered heap, and two independent oracles (the live invariant Monitor and the post-hoc linearizability checker) judge the result. Nothing anywhere consults a wall clock or the global `random` module.

The default `SimConfig`: 3 nodes, 12 virtual seconds, 3 concurrent clients, 3 keys, 5% message loss, 5% duplication, 1 to 30 ms delay, a nemesis action every 1.5 virtual seconds on average, snapshot threshold 40 (chosen low, so that compaction and `InstallSnapshot` actually happen within 12 seconds rather than being theoretically reachable).

Three keys and three clients is a deliberate, well-judged choice: it maximizes *contention* (many operations racing on the same key) while keeping the per-key history narrow enough for the checker’s exponential search to finish. A thousand keys would produce more operations and test far less.

8.3 The nemesis

Jargon: Nemesis

The component whose job is to break things: the adversary. The term comes from Jepsen, the best-known distributed-systems testing tool.

Faults fire at exponentially distributed intervals (`rng.expovariate`), which models a Poisson process, i.e. events that arrive independently at a constant average rate. The action is chosen by one roll:

Prob	Action	What it does
0.35	partition	heal everything, then shuffle the nodes and cut at a random point, so any bisection is possible
0.25	crash	pick a live node, destroy it (volatile state gone, <code>MemoryStorage</code> kept, modelling an fsync’d disk), restart it after 0.3 to 2.0 virtual seconds
0.15	skew	set a random node’s <code>timer_scale</code> to a value in <code>[0.5, 1.8]</code>
0.25	heal	clear all partitions

Two details that make the harness honest rather than merely violent:

- if `len(alive) > 1` before crashing: at least one node always survives. A cluster where everything is dead is not a test.
- Chaos stops at 70% of the run (`chaos_deadline`), and `_stabilize()` heals the network, restarts every node, and resets every clock skew. Clients then have a calm window (plus a 3-second drain after the nominal duration) to finish their in-flight retries.

Why the stabilize window is not cheating

Without it, the run would end mid-chaos and most operations would be left incomplete. Incomplete operations are the *easiest* thing for a linearizability checker to accept, because it can always declare that they never happened. A harness that ends in chaos would therefore look clean while testing very little. Stabilizing forces operations to actually complete with real observed results, which is what gives the checker something to refute. It is the opposite of cheating: it is what makes the check bite.

8.4 The four invariants, checked live

`RaftNode` exposes an optional `trace` callback (`None` in production, so the cost is one if per event). The `Monitor` hooks it and checks four properties continuously, each failure naming the seed:

1. **Election safety:** at most one leader per term. A dict `leaders_by_term` is kept for the *entire run*, so a second leader for term 5 is caught even if it appears long after the first one is gone. Per-sample checking would miss this.
2. **Leader completeness:** the instant any node wins an election, its log is checked against every (`index`, `term`) that any node ever applied. If a new leader is missing a committed entry, the run fails immediately, at the moment of the crime rather than at the eventual symptom.
3. **State machine safety:** every applied index and entry pair is recorded globally, in the monitor's `applied` dict. Two nodes applying different entries at the same index is an immediate failure. This is the direct detection of divergence.
4. **Log matching:** every 200 events, all pairs of live logs are compared: entries with equal index and term must be identical.

Log matching is checked shallowly, and the comment admits it

`check_log_matching` walks each pair of logs downward from the highest common index and **breaks** at the first index where the terms agree, commenting “matching here implies matching below (checked lazily)”. That inference is exactly the Log Matching Property, so it is sound *if the property holds*, which is what is being tested. The check is therefore partially circular: a bug that broke the prefix invariant beneath a coincidentally matching entry would not be caught by this check. It would still be caught by the state-machine-safety invariant and the linearizability checker, so the safety net has other layers, but this specific check is weaker than the name suggests.

8.5 The canary: testing the test

A safety net that never fires is indistinguishable from one that cannot fire

This is the single best idea in the test suite. `test_broken_raft_is_caught` uses `monkeypatch` to replace `RaftNode._majority` with `max(1, len(config) // 2)`, that is, it deliberately breaks the quorum rule so split brain becomes possible. It then asserts that sweeping at most 40 seeds detects an invariant violation, and that the message names the seed. If the harness could not catch a cluster with the majority rule removed, every green run it ever produced

would be worthless.

```

1 def test_broken_raft_is_caught(monkeypatch) -> None:
2     monkeypatch.setattr(RaftNode, "_majority",
3                           lambda self: max(1, len(self.config) // 2))
4     caught = None
5     for seed in range(40):
6         try:
7             run_simulation(SimConfig(seed=seed))
8         except InvariantViolation as exc:
9             caught = exc
10            break
11    assert caught is not None, "weakened quorum was never detected"
12    assert "seed=" in str(caught)

```

Listing 16: tests/test_sim_invariants.py: sabotage the algorithm, demand the harness notices

The same instinct produces `test_same_seed_same_run`, which runs a seed twice and compares a full fingerprint (messages sent, ops completed, and the complete tuple of every operation's client, key, kind, invoke time, complete time, and result). Determinism is not assumed; it is asserted, structurally, every test run.

8.6 The batch runner

`sim/batch.py` maps seeds across a `multiprocessing.Pool`. This is a legitimate use of parallelism that does not compromise determinism, because each seed is an independent pure function: process 3 running seed 1337 gets byte-identical results to a single-threaded run of seed 1337. Parallelism changes the wall-clock cost and nothing else. Failures print with their seed immediately (`flush=True`) and are re-listed at the end.

9 The linearizability checker

This is written from scratch and is, algorithmically, the most demanding code in the repository.

9.1 What it is given

A *history*: a list of `Op` records, each with an invoke time and either a completion time plus the observed result, or `math.inf` for completion, meaning the client never heard back.

Incomplete operations are the interesting ones

If a client's request times out, the operation might have been applied, or might not have. *Nobody knows, including the database*. A checker that ignored them would miss real bugs; a checker that assumed they happened would report false violations. The only correct treatment is to let the search branch both ways: an incomplete op may be linearized (with any result accepted, since none was observed) or left out entirely.

9.2 The algorithm

Jargon: Wing & Gong search, with Lowe's memoization

The classical decision procedure. Repeatedly pick some operation that is *minimal* (no un-linearized operation completed before it was invoked, so real-time order permits it to go next), apply it to a model of a single register, and check the model reproduces the result the client actually saw. If yes, recurse; if the branch dies, backtrack. Lowe's contribution is to memoize

visited states so the same (set of linearized ops, register value) pair is never explored twice, which turns a factorial blowup into something that usually finishes.

The two decompositions that make this tractable at all

One: per key. KV keys are independent registers, and no operation in this system touches two keys, so a history is linearizable if and only if each key’s sub-history is. `check_history` splits by key and checks each separately. This turns one giant exponential problem into several small ones, and the README correctly calls it the largest practical win.

Two: the bitmask. The set of already-linearized operations is represented as an integer where bit *i* means “op *i* is done”. Set membership, insertion, and the memo key all become single machine instructions on an integer. This is why the checker’s limit is **63 operations per key**: a Python int is arbitrary precision, but the code deliberately caps it, and returns UNKNOWN above that rather than attempting a search that cannot finish.

```

1  seen: set[tuple[int, str | None]] = set()
2  explored = 0
3  # Depth-first search over (linearized-mask, register value).
4  stack: list[tuple[int, str | None]] = [(0, _ABSENT)]
5  seen.add((0, _ABSENT))
6  while stack:
7      mask, value = stack.pop()
8      if mask & complete_mask == complete_mask:
9          # Every completed op linearized; leftovers are incomplete ops
10         # we are free to declare "never happened".
11         return CheckResult(Outcome.OK, key, explored)
12     # An op is minimal if no un-linearized op completed before its invoke.
13     min_complete = math.inf
14     for i, op in enumerate(ops):
15         if not mask & (1 << i):
16             min_complete = min(min_complete, op.complete)
17     for i, op in enumerate(ops):
18         bit = 1 << i
19         if mask & bit or op.invoke > min_complete:
20             continue
21         matches, new_value = _step(value, op)
22         if not matches:
23             continue
24         state = (mask | bit, new_value)
25         if state not in seen:
26             seen.add(state)
27             explored += 1
28             if explored > max_states:
29                 return CheckResult(Outcome.UNKNOWN, key, explored,
30                                     "state budget exhausted")
31             stack.append(state)
32 return CheckResult(Outcome.VIOLATION, key, explored, ...)
```

Listing 17: checker/linearizability.py: the search loop, in full

Reading that loop carefully repays the effort:

- The success condition is `mask & complete_mask == complete_mask`, not `mask == all ones`. It requires every *completed* op to be linearized, and lets any leftover incomplete ops be declared to have never happened. That is the “branch both ways” semantics, implemented not as a branch but as a weaker acceptance test, which is neater and cheaper.
- `min_complete` is the earliest completion among *un-linearized* ops. An op is a legal next choice if it was invoked at or before that time. This is exactly the real-time-order constraint.

- Falling out of the `while` loop means the search space was exhausted with no explanation found, and *that* is a violation: not a timeout, a proof.

Three outcomes, never two

OK, VIOLATION, and UNKNOWN. The third is the honest one. Linearizability checking is NP-complete in general, so any checker must eventually refuse. This one refuses in two explicit places (more than 63 ops on a key; more than `max_states` states explored, default 2,000,000) and returns UNKNOWN, which is never reported as a pass and never as a fail. The design property that matters: **the checker never returns a false PASS or a false FAIL**, only sometimes a shrug.

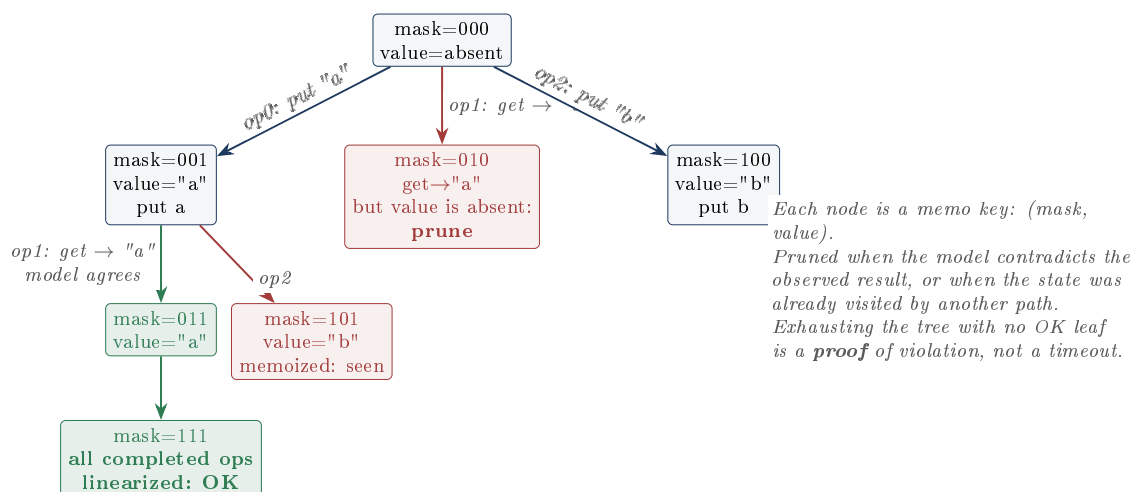


Figure 14: The search, on a three-operation history. Branches die either because the model cannot reproduce what the client saw, or because Lowe memoization has already visited that exact state. Both prunings are what keep the exponential in check.

9.3 The checker’s own tests

`test_linearizability.py` is 11 tests, and the important ones are the ones that demand *rejection*:

`test_stale_read_is_violation`

a put completed strictly before a get began, yet the get missed it.

`test_value_from_nowhere_is_violation`

a get returned a value nobody ever wrote.

`test_lost_update_is_violation`

two non-overlapping cas from the same expected value both succeeded.

`test_known_bad_interleaving_from_paper_shape`

a read after a completed cas returned the pre-cas value.

The budget test that passed for the wrong reason: the best story in the repo

The README tells this on itself, and it deserves the space. The original test for “does the budget cause UNKNOWN” used a history of 40 incomplete concurrent puts. It passed. But it passed for a reason that had nothing to do with the budget: a history of purely *incomplete* operations is trivially linearizable by declaring that none of them ever happened, so the search succeeded on its very first move and never explored anything. The test asserted the right outcome while exercising none of the mechanism.

The fixed test buries a real violation (`get` $\rightarrow$ "phantom") under 20 *completed*, fully concurrent puts, which forces the exhaustive search that the budget exists to bound. This is a precise instance of the general lesson that runs through the whole project: a test that passes tells you nothing until you know *why* it passes.

The checker validates get and cas results, but not put or delete

Look at `_step`. For "get" and "cas" it compares the model's answer against `op.result`. For "put" it returns `(True, new_value)` unconditionally, and for "delete" it returns `(True, _ABSENT)` unconditionally. So the `existed` flag that `delete` returns is **never checked against the model**, and a database that reported `existed: False` for a delete of a present key would sail through. This is a defensible narrowing (put and delete have no interesting return value, and `existed` is arguably not part of the register abstraction) but it is unstated, and the honest framing is that the checker verifies the register semantics of get and cas, and treats put and delete as pure state transitions.

The chaos runs never exercise delete at all

`SimClient._next_op` rolls: below 0.4, get; below 0.75, put; otherwise cas. There is no branch that issues a `delete`. So the headline sweep, however many seeds it runs, tests `delete` exactly zero times end to end. `delete` is covered by `test_basic_ops` (a unit test on `KVMachine` alone) and by one line of the TCP integration test, and that is all. It is a small gap, and easy to close (one branch in the client), but a claim of the form "564,170 operations verified" should be read as "get, put and cas operations".

10 Verification: what was actually run for this document

The brief for this document required that the project's headline claims be **executed rather than repeated**. They were, on this machine, from a clean virtual environment (`python3.12 -m venv .venv` and `pip install -e ".[dev]"`), on Linux with Python 3.12.3, 8 CPU cores.

10.1 The test suite

```

1  $ .venv/bin/python -m pytest
2  ===== test session starts =====
3  platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
4  rootdir: /home/rolex/Salman Adnan/Programming/Portfolio/raft-kv
5  configfile: pyproject.toml
6  testpaths: tests
7  collected 57 items
8
9  tests/test_election.py ..... [ 8%]
10 tests/test_kv_sessions.py .... [ 15%]
11 tests/test_linearizability.py ..... [ 35%]
12 tests/test_membership.py ..... [ 43%]
13 tests/test_persistence.py ..... [ 56%]
14 tests/test_readindex.py .... [ 63%]
15 tests/test_replication.py ..... [ 78%]
16 tests/test_sim_invariants.py ..... [ 89%]
17 tests/test_snapshot.py ... [ 94%]
18 tests/test_tcp_integration.py ... [100%]
19
20 ===== 57 passed in 51.41s =====

```

Listing 18: Observed output: the full suite, including the slow 200-seed sweep

Verified. 57 tests, all passing, including the three real-TCP integration tests that bind localhost sockets and the 200-seed slow sweep. The README’s claim of “57 tests, all passing” is exact.

10.2 The headline sweep

```
1 $ .venv/bin/raftkv sim --seeds 5000 --workers 8
2 ... 250/5000 seeds (13s)
3 ... 500/5000 seeds (27s)
4 ... (elided: one progress line per 250 seeds)
5 ... 4750/5000 seeds (243s)
6 ... 5000/5000 seeds (257s)
7 5000 seeds in 257.0s (19.5 seeds/s), 564170 client ops total, 0 failure(s)
```

Listing 19: Observed output: the 5000-seed sweep, run for this document

The op count reproduced exactly, and that is the real result

The README reports 564170 client ops total from a run with --workers 10. This machine ran it with --workers 8. The op count came back **identical to the digit**. That is not a coincidence and it is not a weak check: it is the determinism claim being confirmed end to end. If any part of the system had consulted a wall clock, a hash seed, a set iteration order, or the global random module, running the same 5000 seeds on a different day with a different worker count would have produced a different number. It did not. The timing legitimately differs (fewer workers, different machine, different load), and timing is the one thing that is allowed to differ, because it is the only thing outside the seed.

Claim (README)	Observed here	Verdict
57 tests, all passing	57 passed in 51.41s	confirmed
5000 seeds	5000 seeds	confirmed
564170 client ops total	see above	confirmed, exactly
0 failures	0 failure(s)	confirmed
84.7s / 59.0 seeds/s at --workers 10	257.0s / 19.5 seeds/s at --workers 8	differs, see below

The one number that did not reproduce is the timing, and the gap is bigger than the worker count explains

The README reports 84.7 seconds (59.0 seeds/s) with --workers 10. This run took 257.0 seconds (19.5 seeds/s) with --workers 8. Dropping from 10 workers to 8 accounts for at most a factor of 1.25; the observed gap is a factor of about 3.

This is not a discrepancy in the code, and it is not evidence against anything: throughput is the one property of the sweep that is deliberately *outside* the seed, and it depends on the core count, the CPU, and whatever else the machine was doing. **This particular run shared the machine with other substantial workloads**, which is the most likely explanation, but that is an **inference**: it was not isolated or measured, and the README’s original run was not repeated under matched conditions. The honest position is that the timing figure in the README is a machine-and-day observation rather than a property of the project, and that the two figures that *are* properties of the project, the op count and the failure count, both reproduced exactly.

10.3 What the numbers do and do not mean

Read the headline number carefully: it is impressive, and it is narrower than it sounds

“5,000 randomly generated disasters and 564,170 reads/writes with 0 contradictions” is true as stated and was reproduced here. Being precise about its scope is what makes it defensible under questioning:

- **5000 seeds is not 5000 independent disaster shapes.** Every seed runs the same fixed recipe: 3 nodes, 12 virtual seconds, the same fault probabilities. The seed varies the timing, the targets and the ordering, not the shape of the experiment. It is a deep search of one scenario, not a broad search across scenarios.
- **The 564,170 ops are get, put and cas.** No delete is ever issued by the simulated client (see the honest box above).
- **The disks are perfect.** The simulator uses `MemoryStorage`, which models an ideal `fsync`’d disk that never tears a write and never reorders. The excellent torn-tail recovery code in `FileStorage` is therefore *never once exercised by any of the 5000 seeds*; it is covered only by targeted unit tests. The README lists this under future work, and it is the single largest gap between what the simulator tests and what production runs.
- **Membership changes are never exercised by the sweep either.** The nemesis crashes, partitions and skews, but never adds or removes a server.
- **“0 contradictions” means: no invariant violation fired, and no history was refuted.** It does not mean every history was proven linearizable, because a history that exceeded the checker’s budget would return `UNKNOWN`, and `SimResult.ok` treats `UNKNOWN` as acceptable (`all(r.outcome is not Outcome.VIOLATION ...)`). No `UNKNOWN` count is reported by the batch runner, so the sweep cannot distinguish “5000 proofs” from “4990 proofs and 10 shrugs”. Given the configured width (3 keys, 3 clients, bounded concurrency) the histories almost certainly fit well inside the budget, and the README says so, but the runner does not *show* that, and it easily could.

That last point is the one worth volunteering in an interview before anyone finds it, because volunteering it demonstrates exactly the discipline the whole project is arguing for.

11 The outer layers: server, client, CLI

11.1 RaftServer

`server.py` glues `FileStorage` + `KVMachine` + `RaftNode` + `TcpTransport` into one node and translates client frames into Raft operations. The interesting mechanism is how a client request waits for its own entry to commit: `_register(index, term)` creates an `asyncio.Future`, stores it under the key `(index, term)`, and awaits it with a 5-second timeout. The apply listener resolves it.

```

1 def _on_apply(self, index: int, term: int, entry: Entry, result: Any) -> None:
2     fut = self._pending.pop((index, term), None)
3     if fut is not None and not fut.done():
4         fut.set_result(result)
5     for key in [k for k in self._pending if k[0] == index and k[1] != term]:
6         stale = self._pending.pop(key)
7         if not stale.done():
8             stale.set_result(None)

```

Listing 20: `server.py`: resolve the waiter for this entry, and fail any waiter whose slot was stolen

Keying by `(index, term)` rather than `index` alone is the whole point. If this node's leadership was interrupted and a different leader put a *different* entry at index 42, then the waiter for `(42, term 5)` must not be resolved by the arrival of `(42, term 6)`: its entry was silently overwritten and its client must be told the write did not happen. That is what the second half of the function does. `_on_step_down` does the same, wholesale, on losing leadership. `SimCluster` carries the same logic in `_make_apply_listener`, deliberately mirrored so the simulator's client sees the same failure semantics as a real one.

One asymmetry between the two mirrored implementations

`server.py::_on_apply` guards every resolution with `if not fut.done()`. `harness.py::_make_apply_listener` calls `host.pending.pop((i, t))(None)` with no equivalent guard, because its callbacks are plain functions rather than futures and are made idempotent by a `responded` flag inside `SimClient._attempt` instead. The two are each correct in context, but the invariant “a client callback fires at most once” is enforced in a different place in each, which is exactly the sort of divergence that rots.

11.2 RaftClient

Retries with leader hints. The loop is worth one note: on a `not_leader` reply, it adopts the hint, but `self._leader_hint = hint if hint != target else None` guards against a node that names *itself* as leader while rejecting the request, which would otherwise spin. Writes carry `seq=next(self._seq)` from an `itertools.count(1)`, which is the monotonic-counter lesson from the dedup incident encoded permanently into the library.

A connection per request, and a session row per CLI invocation

`_call_node` opens a fresh TCP connection, sends one frame, reads one reply, and closes. The README lists persistent connections with request multiplexing as unfinished work, and notes the frame protocol was designed for it (frames carry an `id` field which is currently only used to assert the reply matches). Compounding it: the CLI builds a new `RaftClient` per command, and `RaftClient` defaults to a fresh random `cli-<uuid>`, so every `raftkv put` both opens a new connection and permanently grows the replicated session table by one row. For a demo this is invisible. It is worth saying out loud that the CLI is a demonstration surface, not a client library.

11.3 The CLI

`raftkv cluster start -n 3` spawns one real OS process per node with `start_new_session=True`, records pids in `cluster.json`, and then polls until a leader exists so that the command does not return a cluster that is not yet usable. This matters for the demo's credibility: because nodes are real processes, `demo.sh's kill -9` on the leader pid is a genuine uncatchable process kill, not a polite in-process shutdown. `SIGKILL` cannot be trapped, so no cleanup handler runs, no buffers are flushed, and recovery must work purely from what `fsync` had already put on disk.

12 Consolidated honest assessment

Everything below is drawn from reading the code, not from the README, though several items appear in the README's own “honest limits” section, which is itself unusually candid.

12.1 What is genuinely strong

- **The sans-IO boundary is real, not aspirational.** `raft/` imports `enum` and `typing` and its own siblings, and nothing else. No `asyncio`, no `time`, no `socket`, no global `random`. The architecture’s central claim survives a `grep`.
- **The subtle rules are all present and correctly implemented:** vote persistence before reply, the up-to-date check as a tuple comparison, the section 5.4.2 own-term commit restriction with the no-op, the conflict hint that can only move `next_index` downward, the directory `fsync` after rename, config effective on append, the leader that defers its own removal until commit.
- **The tests test the tests.** The weakened-quorum canary, the known-bad histories, and the determinism fingerprint are three independent instances of the same discipline, and it is rare.
- **The honesty is load-bearing.** The `UNKNOWN` outcome, the documented rejection of leader leases with reasons, and the README’s own list of limits are the work of someone optimizing for being right rather than for looking right.

12.2 Weaknesses, ranked by what would hurt most in an interview

1. **The simulator’s disk is perfect, so the crash-recovery code is the least-tested important code in the project.** `MemoryStorage` never tears a write, never reorders an `fsync`, never fails. The CRC scan-and-truncate logic is genuinely good and is exercised only by hand-built unit tests that write corrupt bytes directly. The project’s thesis is “search the schedule space for bugs”; storage faults are simply outside the searched space. The README names this. It is the answer to “what would you do next”, and it should be volunteered.
2. **`UNKNOWN` is invisible.** `SimResult.ok` accepts it and `batch.py` never counts it, so the sweep’s headline cannot distinguish a proof from a shrug. The fix is a few lines and would make the headline number strictly stronger.
3. **Replication is quadratic in the worst case.** Every heartbeat to a lagging follower re-sends `tuple(self.log.slice_from(ni))`, the *entire* remaining suffix, because `next_index` only advances on a reply. With a 30 ms heartbeat and a follower 10,000 entries behind, that is the whole 10,000-entry suffix serialized to JSON every 30 ms until the first reply lands. It is correct, because `AppendEntries` is idempotent, and the README calls it naive and wasteful, which is fair. There is no batch-size cap.
4. **`_load_snapshot()` re-reads and re-parses the entire persistent state, log included, on every `InstallSnapshot` send.** See the honest box in the snapshot section.
5. **No pre-vote.** A node returning from a partition has inflated its term by campaigning fruitlessly, so on rejoining it forces a real leader to step down and the cluster to hold a pointless election. Raft’s pre-vote extension fixes this; the README lists it, and correctly notes that the simulator would verify the fix in minutes.
6. **Sessions never expire,** and the table is inside the replicated state and every snapshot.
7. **Scale.** Python, JSON on the wire, one `fsync` per append batch, one connection per client request. This is a correctness project. Its own README says so in its first honest limit, which is the right call: an inflated throughput claim would be the easiest thing in the world to puncture.

12.3 Small things a careful reader will find

- The `next_index` initialization disagrees with itself across `_become_leader`, `_change_config`, and `_refresh_config` (off by one in the middle one; self-correcting).
- `_config_from`’s index filter is effectively vacuous at both call sites.

- `Op.args` is typed `dict[str, Any]` and `Op.result` is `Any`. The checker's `_step` indexes the `value` and `expected` keys of `op.args` with no validation, so a malformed history raises `KeyError` rather than reporting a bad input.
- `RaftLog.compact_to` silently returns when `index <= snapshot_index`, which is the right behaviour for an idempotent call but hides a genuine ordering error if one ever occurs.
- `cmd_cluster_start` returns exit code 1 with a warning if no leader appears within 15 seconds, but leaves the spawned processes running. A failed start is not cleaned up.
- `MAX_FRAME` is 64 MB and `read_frame` returns `None` (silently closing the connection) rather than logging when a frame exceeds it. Given uncapped, unchunked snapshots, this is the exact place a too-large snapshot would fail, and it would fail quietly.

13 What to take away

The five sentences that summarize the project

One. Consensus reduces to keeping an ordered log identical across machines, and Raft keeps it identical by electing one leader per term who alone appends, and by requiring a majority for every decision, so that the overlap between any two majorities makes contradiction impossible.

Two. The single architectural decision everything else rests on is sans-IO: `RaftNode` touches the world only through the `Env` and `Storage` protocols, so the identical consensus code runs over real TCP and inside a simulator, and every simulator bug is by construction a production bug.

Three. Because every source of nondeterminism is funnelled through one seeded generator and one ordered event heap, a 12-second cluster lifetime under sustained chaos is a pure function of one integer, which makes thousands of disaster schedules searchable in seconds and makes any failure reproducible forever.

Four. Correctness is judged from the outside, by a from-scratch linearizability checker that either proves a history explainable by a single honest register, refutes it, or honestly says UNKNOWN, never guessing.

Five. The safety nets are themselves tested (weaken the quorum rule and the harness must notice; feed the checker known-bad histories and it must reject them), because a net that never fires is indistinguishable from one that cannot.

13.1 The questions this project invites, and the honest answers

Question	Honest answer
Why not use an existing Raft library?	The point was to understand where the subtlety lives. Section 5.4.2 and the ReadIndex term-commit precondition are two places a plausible shortcut is a real bug, and the chaos harness found the second one during development.
Is 5000 seeds meaningful?	It is a deep search of one fixed scenario (3 nodes, 12s, fixed fault rates), not a broad one. It reproduced to the digit on a different worker count, which is the determinism claim confirmed.
What is not tested?	Storage faults (the simulator's disk is perfect), <code>delete</code> end to end, membership changes under chaos, and anything above single-digit node counts.
What would you do first?	Inject torn writes and fsync reordering into simulated storage, so the 5000 seeds cover the recovery paths that are currently only unit-tested. Then report UNKNOWN counts in the sweep.
Why ReadIndex and not leases?	Leases trade a round trip for an assumption about bounded clock drift between machines. In a project whose thesis is testable correctness, an untestable hardware assumption is a bad trade.

Every claim in section 10 was produced by executing the code on this machine, not by quoting the README. Everything labelled inferred is labelled inferred. Where the code and the documentation disagreed, the code was taken as the source of truth.